TOPIC 9

Task 1

```
import Foundation
struct Student {
    let name: String
    let marks: Int
    let dept: String
}
let students = [
    Student(name: "Asha", marks: 82, dept: "IT"),
    Student(name: "Ravi", marks: 35, dept: "CSE"),
    Student(name: "Meena", marks: 91, dept: "IT"),
    Student(name: "Kiran", marks: 56, dept: "ECE"),
    Student(name: "Divya", marks: 67, dept: "CSE"),
    Student(name: "Arjun", marks: 40, dept: "IT")
]
let passed = students.filter { $0.marks >= 40 }
print("Passed Students:")
for s in passed {
    print(s.name, s.marks)
}
let names = students.map { $0.name }
print("Names:", names)
let total = students.map { $0.marks }.reduce(0, +)
let average = Double(total) / Double(students.count)
print("Average:", average)
let ranked = students.sorted { $0.marks > $1.marks }
print("Rankings:")
for s in ranked {
```

```
    print(s.name, s.marks)
}
let groups = Dictionary(grouping: students, by: { $0.dept })
print("Departments:")
for dept in groups.keys.sorted() {
    print(dept, groups[dept]!.map { $0.name })
}
let result1 = students.filter { student in
    return student.marks >= 40
}
let result2 = students.filter { $0.marks >= 40 }
let studentNames = students.map(\.name)
print(studentNames)
let highMarks = students.compactMap {
    $0.marks >= 60 ? $0.marks : nil
}
print("Marks above 60:", highMarks)
```

OUTPUT:

```
Passed Students:
Asha 82
Meena 91
Kiran 56
Divya 67
Arjun 40
Names: ["Asha", "Ravi", "Meena", "Kiran", "Divya", "Arjun"]
Average: 61.833333333333336
Rankings:
Meena 91
Asha 82
Divya 67
Kiran 56
Arjun 40
Ravi 35
Departments:
CSE ["Ravi", "Divya"]
ECE ["Kiran"]
IT ["Asha", "Meena", "Arjun"]
["Asha", "Ravi", "Meena", "Kiran", "Divya", "Arjun"]
Marks above 60: [82, 91, 67]
```

Task 2:

```
func myFilter(_ numbers: [Int], _ check: (Int) -> Bool) -> [Int] {
    var result: [Int] = []
    for n in numbers {
        if check(n) {
            result.append(n)
        }
    }
    return result
}
let numbers = [1, 2, 3, 4, 5, 6]
print(myFilter(numbers) { $0 % 2 == 0 })
func applyTwice(_ value: Int, _ f: (Int) -> Int) -> Int {
    return f(f(value))
```

```
}
print(applyTwice(3) { $0 * 2 })
class Button {
    var handlers: [() -> Void] = []
    func onTap(_ action: @escaping () -> Void) {
        handlers.append(action)

    func tap() {
        for action in handlers {
            action()
        }
    }
}
let button = Button()
button.onTap {
    print("Button clicked")
}
button.onTap {
    print("Welcome")
}
button.tap()
```

OUTPUT:

```
[2, 4, 6]
12
Button clicked
Welcome
```

Task 3:

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1
        return count
    }
}
let a = makeCounter()
let b = makeCounter()
print(a(), a(), b(), a())
```

OUTPUT:

```
1 2 1 3
```